

Code Description — Mathematical Logic, Design Reasoning, Verification, and Code Lineage

1. Why I wrote this document

I wrote this document so that I can explain the computational side of my research clearly to a mathematically trained reader. My aim is not simply to say what each Python file does. I want to explain **why I chose each computational idea, what mathematical object it operates on, what equations it implements, what assumptions it makes, why I did not choose an apparently simpler alternative, and how one verified component became the basis for a later component.**

I deliberately distinguish between three things:

1. **Mathematical principles** — e.g. blind source separation, covariance geometry, Riemannian distance, Fréchet means, tangent-space representation, robust statistics, LOSO validation.
2. **Engineering choices** — e.g. filter cut-offs, MAD thresholds, window lengths, learning rates, feature combinations.
3. **Verification mechanisms** — known-answer tests, finite-difference gradients, negative controls, permutation/bootstrap tests, cross-subject validation, and independent human-EEG validation.

The repository’s `SCRIPT_INDEX.md` defines the load-bearing scripts and explicitly separates them from the exploratory/history directory. I use that distinction here: I do **not** describe the contents of `exploratory_not_required_for_paper/` as part of the main computational chain. The canonical reproduction order is Stage 1 $\rightarrow$ Stage 2 $\rightarrow$ Stage 3 $\rightarrow$ Stage 4, followed by the statistical and narration guardrails where applicable.

2. The research question behind the code

My computational question is not simply:

“Can I obtain a high BCI accuracy?”

My stronger question is:

Can I construct a low-channel EEG/BCI computational pipeline in which the mathematical transformations, source separation, covariance representation, machine-learning evaluation, streaming behavior, and final reported claims can each be independently checked?

That is why the code evolved as a chain of tests rather than as one monolithic classifier.

The overall mathematical flow is:

$$X(t) \rightarrow F\{X(t)\} \rightarrow \text{source separation} \rightarrow C \rightarrow \mathcal{S}_{++}^n \rightarrow M_R \rightarrow T_C \rightarrow \text{ML} \rightarrow \text{subject-level evaluation}.$$

Here:

- $X(t)$ is the observed multichannel EEG,
- F is signal preprocessing,
- source separation attempts to recover latent components,
- C is a spatial covariance matrix,

- $\mathcal{S}_{++}^n$ is the manifold of symmetric positive-definite matrices,
- M_R is a Riemannian/Fréchet mean,
- T_C is a tangent-space representation,
- and the final ML layer operates either directly on engineered features or on tangent-space features.

The important design principle is that **I verify a mathematical layer before trusting it downstream.**

3. Stage 1 — establish mathematical correctness before biological interpretation

Stage 1 contains controlled synthetic experiments. I use synthetic data because I know the ground truth. If an algorithm fails here, I do not need to debate whether the failure came from complicated biological EEG.

The key files are:

- `sobi.py`
- `heuristic_sensitivity.py`
- `bootstrap_and_permutation.py`
- `proper_stats.py`
- `ar4_benchmark.py`
- `mini_eegnet.py`
- `riemannian.py`
- `phase2_sqi.py`
- `phase3_streaming_ola.py`
- `phase4_tournament.py`
- `four_way_tournament.py`
- `phase4_noise_sweep.py`
- `rms_scaled_injection.py`

3.1 `sobi.py` — my source-separation mathematical primitive

What I wanted to test

I wanted a transparent implementation of Second-Order Blind Identification (SOBI), rather than beginning with a black-box source-separation library.

The assumed model is:

$$X = SA^T,$$

where S contains latent sources and A is the unknown mixing matrix.

The objective is to estimate an unmixing matrix W such that:

$$S \approx X_c W^T,$$

where X_c is the mean-centered observation.

Step 1: whitening

I calculate the zero-lag covariance:

$$C_0 = \frac{X_c^T X_c}{N}.$$

I eigendecompose it:

$$C_0 = E \Lambda E^T.$$

Then I construct:

$$W_h = \Lambda^{-1/2} E^T,$$

and whiten the data:

$$Z = X_c W_h^T.$$

Whitening matters because it converts the unknown instantaneous mixing problem into an orthogonal rotation problem. After whitening, the remaining source-separation transformation can be represented by an orthogonal matrix.

Step 2: time-lagged covariance

SOBI does not rely primarily on non-Gaussianity. It uses temporal dependence.

For each lag τ , I compute:

$$R_\tau = \frac{1}{N - \tau} Z_{1:N-\tau}^T Z_{\tau+1:N}.$$

I symmetrize it numerically:

$$R_\tau \leftarrow \frac{R_\tau + R_\tau^T}{2}.$$

I use multiple lags, currently 20.

Step 3: approximate joint diagonalization

I seek an orthogonal V for which:

$$V^T R_\tau V \approx D_\tau$$

for all selected lags.

The implementation uses Jacobi rotations. For each coordinate pair (p, q) , it builds the two-dimensional criterion from diagonal differences and off-diagonal terms, forms $G = g^T g$, chooses the eigenvector associated with the smallest eigenvalue, and converts it into a rotation angle:

$$\theta = \frac{1}{2} \text{atan2}(x, y).$$

The corresponding rotation is repeatedly applied until the residual off-diagonal energy is sufficiently small.

Why Jacobi rotations?

I wanted a deterministic and inspectable joint-diagonalization procedure. A generic nonlinear optimizer would introduce another layer of numerical optimization, initialization, stopping criteria, and possible local behavior. Jacobi rotations directly express the diagonalization objective.

Why SOBI rather than only FastICA?

FastICA exploits non-Gaussianity. SOBI exploits temporal structure. EEG is temporally structured, so SOBI is a meaningful comparator rather than merely another classifier.

The later benchmark therefore asks a principled question:

Does exploiting temporal second-order structure behave differently from exploiting non-Gaussianity under the same controlled artifact-removal task?

Verification lineage

I had a Jacobi-rotation sign error earlier. The known-answer test exposed it. The corrected version recovers the known mixed sinusoidal sources at approximately 1.0000 correlation. This became important later: when I reused SOBI in the streaming pipeline, I did **not** implement another unverified SOBI. `phase3_streaming_ola.py` explicitly imports this verified implementation.

That is an important dependency relationship in my project:

verify SOBI once → reuse the same verified SOBI downstream

Source: `stage1_signal_processing/sobi.py`.

3.2 `heuristic_sensitivity.py` — controlled FastICA vs. SOBI benchmark

I construct three deterministic neural-like oscillations:

6 Hz, 10 Hz, 20 Hz

representing theta, alpha, and beta, plus Gaussian background noise.

I mix them through a known matrix A :

$$X = SA^T.$$

Then I inject two controlled artifacts:

- a blink-like transient using a Hann envelope,
- a muscle-noise interval.

I filter the result to 1–30 Hz and run both FastICA and my verified SOBI implementation.

For artifact identification I intentionally test two simple heuristics:

Maximum-amplitude component

$$i^* = \arg \max_i \max_t |S_i(t)|.$$

Maximum-kurtosis component

$$i^* = \arg \max_i |\text{kurtosis}(S_i)|.$$

I then zero the selected latent component and reconstruct the sensor-space signal.

Why these heuristics?

This is not intended to claim that maximum amplitude or kurtosis is a universal EEG artifact classifier. The purpose is controlled sensitivity analysis: if the separation algorithm places the designed artifact into a distinct latent component, simple component-level rules should be able to expose it.

The metrics are correlation and SNR:

$$\text{SNR} = 10 \log_{10} \frac{\sum x_{clean}^2}{\sum (x_{test} - x_{clean})^2}.$$

The result is the synthetic benchmark reported in the paper.

The repository later corrected its documentation so that this file, rather than `phase1_diagnostic.py`, is identified as the source of the synthetic FastICA/SOBI result. This is an example of repository-level provenance correction: I do not want a plausible filename to be mistaken for the actual result-generating script.

Source: `stage1_signal_processing/heuristic_sensitivity.py`.

3.3 `rms_scaled_injection.py` — correcting an artifact-amplitude mismatch

The purpose of this small utility is to make synthetic artifacts relative to the signal amplitude of each channel.

For each channel:

$$RMS_c = \sqrt{\frac{1}{N} \sum_t x_c(t)^2}.$$

Blink amplitude is then:

$$A_{blink} = 10 RMS_c,$$

and the muscle-noise scale is:

$$A_{muscle} = 4.3 RMS_c.$$

Why did I need this?

Fixed absolute artifact amplitudes can accidentally make one channel unrealistically contaminated while another is barely affected. Scaling to each channel's own baseline makes the injection relative to the signal being perturbed.

This is a small but important lesson in synthetic benchmarking: the artifact generator itself must not introduce an artificial advantage through arbitrary amplitude units.

Source: `stage1_signal_processing/rms_scaled_injection.py`.

3.4 `bootstrap_and_permutation.py` — uncertainty rather than a single number

This script adds statistical context to previously computed results.

Bootstrap correlation interval

For paired series x, y , I resample indices with replacement and recompute Pearson correlation:

$$r = \text{corr}(x, y).$$

After 2,000 bootstrap resamples, I take the empirical 2.5th and 97.5th percentiles as a 95% interval.

Paired permutation test

For the ML tournament, I calculate per-sample squared errors:

$$e_{RF,i} = (y_i - \hat{y}_{RF,i})^2,$$

$$e_{Ridge,i} = (y_i - \hat{y}_{Ridge,i})^2.$$

The paired difference is:

$$d_i = e_{Ridge,i} - e_{RF,i}.$$

I then randomly flip the sign of each paired difference and recompute the mean many times. This tests whether the observed model difference is distinguishable from a null in which the direction of each paired advantage is arbitrary.

Why not use only a t-test?

Because I do not want to assume normality unnecessarily for these small synthetic comparisons. Bootstrap and permutation methods are direct empirical procedures and fit the paired nature of the comparison.

Source: `stage1_signal_processing/bootstrap_and_permutation.py`.

3.5 `proper_stats.py` — repeated synthetic realizations and frozen-prediction statistics

This script addresses two different questions.

FastICA/SOBI between-recording variability

Instead of trusting one random realization, it repeats the same generator over 200 seeds and reports:

- mean,
- standard deviation,
- 2.5–97.5 percentile range,
- fraction of runs where FastICA exceeds SOBI.

This separates an algorithmic effect from a lucky/unlucky random draw.

CNN vs Random Forest statistical comparison

It first generates **frozen out-of-fold predictions** once. It then performs bootstrap and permutation analysis on those predictions.

This is important because repeatedly retraining a neural network thousands of times during a statistical test would mix two sources of variation:

1. model-training randomness,
2. the statistical resampling procedure.

By freezing the predictions first, the bootstrap is asking a cleaner question about the observed sample-level predictions.

Source: `stage1_signal_processing/proper_stats.py`.

3.6 ar4_benchmark.py — challenging the pure-sine assumption

A pure sine wave is easy to understand but not a realistic stochastic model of EEG.

So I created AR(4) sources.

The process is:

$$x_t = \sum_{k=1}^4 a_k x_{t-k} + \epsilon_t.$$

I construct the coefficients by placing complex-conjugate poles around a desired frequency. For a pole radius r and angular frequency ω , the AR coefficients are obtained from the characteristic polynomial.

This produces resonant, stochastic, band-limited activity instead of perfectly deterministic sinusoids.

I then repeat the FastICA/SOBI comparison.

Why AR(4)?

It is still transparent enough to inspect mathematically, but it removes the strongest criticism of the earlier benchmark: that perfect sinusoids are too idealized.

The result is important because the FastICA/SOBI difference shrinks to less than about one percentage point in the harder stochastic benchmark. This supports the later decision not to claim that SOBI is universally superior to FastICA.

Source: `stage1_signal_processing/ar4_benchmark.py`.

3.7 mini_eegnet.py — verifying a from-scratch neural network before trusting it

I implemented a deliberately small 1-D CNN:

Conv1D(4 → 8, kernel=25)

↓

ReLU

↓

Global Average Pooling

↓

Dense(8 → 1)

The convolution computes, conceptually:

$$y_{t,o} = \sum_{k,c} x_{t+k,c} W_{k,c,o} + b_o.$$

ReLU is:

$$\text{ReLU}(x) = \max(0, x).$$

Global average pooling is:

$$h_j = \frac{1}{T} \sum_t x_{t,j}.$$

The final scalar prediction is linear:

$$\hat{y} = Wh + b.$$

Training uses squared error:

$$L = (\hat{y} - y)^2,$$

with derivative:

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y).$$

The key verification

Because I implemented backpropagation myself, I do not trust a plausible training curve as proof that the gradients are correct.

I compare the analytical gradient with a central finite difference:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + h) - L(\theta - h)}{2h}.$$

The script uses $h = 10^{-5}$ and requires the maximum relative error to be below 10^{-3} . The reported verified run reaches approximately 4.35×10^{-10} .

Why not just use PyTorch/TensorFlow?

For the final research system, established frameworks are sensible. But for this verification stage, the point was to demonstrate that I understood and could independently verify the gradient machinery. The same principle was used for SOBI: implement the mathematical core explicitly, then test it before using it downstream.

Source: `stage1_signal_processing/mini_eegnet.py`.

3.8 `riemannian.py` — moving covariance matrices onto their natural geometry

This is the mathematical foundation of the later Riemannian BCI pipeline.

For each EEG window I calculate a spatial covariance matrix:

$$C = \frac{X_c^T X_c}{N} + \epsilon \frac{\text{tr}(C)}{n} I.$$

The small regularization term helps maintain positive definiteness.

The matrices therefore belong to:

$$\mathcal{S}_{++}^n.$$

Matrix functions

For an SPD matrix:

$$C = U \Lambda U^T,$$

I define:

$$C^{1/2} = U \Lambda^{1/2} U^T,$$

$$C^{-1/2} = U \Lambda^{-1/2} U^T,$$

$$\log C = U (\log \Lambda) U^T,$$

and similarly for the exponential.

Affine-invariant distance

The distance used is:

$$d_R(A, B) = \left[\sum_i \log^2 \lambda_i (A^{-1/2} B A^{-1/2}) \right]^{1/2}.$$

This respects the multiplicative geometry of covariance matrices rather than treating them as arbitrary Euclidean vectors.

Riemannian/Fréchet mean

I seek:

$$M_R = \arg \min_M \sum_i d_R^2(M, C_i).$$

The iterative update is:

$$M_{k+1} = M_k^{1/2} \exp \left[\frac{1}{N} \sum_i \log(M_k^{-1/2} C_i M_k^{-1/2}) \right] M_k^{1/2}.$$

Tangent-space representation

For reference mean M :

$$T_i = \log(M^{-1/2} C_i M^{-1/2}).$$

I vectorize the symmetric matrix, multiplying off-diagonal entries by $\sqrt{2}$ so that the Euclidean vector inner product preserves the matrix Frobenius geometry.

Why not simply average covariance matrices?

The Euclidean mean

$$M_E = \frac{1}{N} \sum_i C_i$$

does not respect the affine-invariant geometry. The later STEW experiment tests this distinction on real human EEG rather than leaving it as a purely theoretical argument.

Verification

I test the simplest known-answer condition: if every input covariance is exactly the same SPD matrix C , the Riemannian mean must return C , and the tangent-space vector of C at its own mean must be zero.

This is the second major mathematical primitive that is verified before downstream use.

Source: `stage1_signal_processing/riemannian.py`.

3.9 phase2_sqi.py — synthetic signal-quality verification

The purpose is to test whether a lead-off/detachment event can be detected quickly.

I combine three checks:

1. amplitude saturation,
2. high-frequency power ratio,
3. normalized spectral entropy.

Spectral entropy

Using a Welch PSD $P(f)$, I normalize the spectral probabilities:

$$p_i = \frac{P_i}{\sum_j P_j}.$$

Then:

$$H = - \sum_i p_i \log_2 p_i.$$

I normalize it by the maximum possible entropy:

$$H_{norm} = \frac{H}{\log_2 N}.$$

Flat thermal noise should have high entropy, while structured oscillatory EEG should have lower entropy.

The high-frequency ratio is:

$$r_{noise} = \frac{P_{35-125Hz}}{P_{1-30Hz}}.$$

I construct an SQI from the ratio and entropy and use the minimum of the two quality contributions.

The synthetic signal is clean before $t = 6s$ and becomes high-amplitude white noise afterward. The validation gate requires SQI to fall below 0.20 shortly after detachment.

Why synthetic data here?

Because I know exactly when the electrode becomes disconnected. This makes the latency claim testable without ambiguity.

Source: `stage1_signal_processing/phase2_sqi.py`.

3.10 phase3_streaming_ola.py — taking the verified SOBI primitive into streaming

This script reuses the verified `sobi.py` rather than creating a second source-separation implementation.

The architecture is:

Calibration window
↓
SOBI once
↓
fixed W_{cal} and A_{cal}
↓
2-second ring buffer
↓
200-ms step
↓
project → remove calibrated artifact component → reconstruct

The central design decision is **calibrate once**.

If I estimated SOBI independently in every 200-ms update, the unmixing matrix could change due to component permutation and finite-sample instability. Instead:

$$W_{cal} = W(X_{cal}),$$

and each new window uses:

$$S_t = (X_t - \bar{X}_t)W_{cal}^T.$$

The identified artifact component is zeroed and reconstructed:

$$\hat{X}_t = S_t^{clean}A_{cal}^T + \bar{X}_t.$$

The code uses a 2-second window and 200-ms update interval. Processing time is compared with a 50-ms gate.

Batch vs streaming

I calculate a batch-cleaned reference using the same verified SOBI and compare:

- streaming vs ground truth,
- batch vs ground truth,
- streaming vs batch.

This separates algorithmic correctness from streaming implementation effects.

Important terminology

The implementation uses overlap averaging as an honest stand-in for a full textbook weighted Hanning overlap-add reconstruction. I therefore treat it as an overlap-based streaming reconstruction benchmark, not as a claim that I implemented every detail of a production OLA filter bank.

Source: `stage1_signal_processing/phase3_streaming_ola.py`.

4. Stage 1 machine-learning tournament

4.1 `phase4_tournament.py`

This script asks whether a small from-scratch CNN is actually better than classical models under the controlled low-channel setting.

I generate 50 synthetic sessions. A continuous latent focus variable controls the amplitudes of theta, alpha and beta sources. A self-report score is generated as a noisy discretized function of that latent focus.

The engineered feature vector contains:

- 4 channels $\times$ 3 band powers = 12 features,
- 6 pairwise channel correlations,
- total = 18 features.

Band powers use Welch's PSD and integration over:

$$\theta : 4 - 8Hz, \quad \alpha : 8 - 13Hz, \quad \beta : 13 - 30Hz.$$

The models are:

- Ridge regression,
- Random Forest regression,
- MiniEEGNet on raw voltage.

Evaluation uses 5-fold cross-validation.

The metrics are:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

and Pearson correlation.

Why compare these models?

I wanted to separate three ideas:

1. a linear classical baseline,
2. a nonlinear classical model,
3. a neural model operating directly on raw time series.

If the CNN wins, that is informative. If it does not, that is also informative. The goal is not to force deep learning to win.

Negative control

The labels are randomly permuted. The expected behavior is approximately zero correlation and negative R^2 when the model cannot generalize to the randomized relationship.

Before trusting the CNN, the script calls `numerical_gradient_check()` from `mini_eegnet.py`.

This creates another explicit chain:

gradient verified → CNN trained → CNN compared against classical models → negative control

Source: `stage1_signal_processing/phase4_tournament.py`.

4.2 `four_way_tournament.py` — adding the Riemannian representation

This extends the tournament to four tracks:

1. Random Forest on engineered Euclidean features,
2. Ridge on engineered Euclidean features,
3. MiniEEGNet on raw voltages,
4. Riemannian tangent-space + Ridge.

Before running, it calls the `riemannian.py` sanity check. This is intentional: the Riemannian feature path is not trusted simply because the code executes.

For each training fold, the Riemannian mean is fitted using **training covariances only**:

$$M_{train} = \text{FrechetMean}(\{C_i : i \in train\}).$$

Both train and test covariances are then mapped using the training reference.

This avoids test-set information entering the geometric reference.

The Riemannian regression model is Ridge:

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|^2 + \lambda \|\beta\|^2.$$

Why Ridge after tangent mapping?

The tangent representation gives Euclidean coordinates, but the number of features can still be large relative to a small synthetic sample. Ridge provides controlled L2 regularization rather than letting the model freely fit every coordinate.

Source: `stage1_signal_processing/four_way_tournament.py`.

4.3 `phase4_noise_sweep.py` — testing robustness rather than clean accuracy

A clean-distribution model comparison is not enough for a BCI system.

I therefore add Gaussian noise with:

$$\sigma \in \{0, 0.3, 0.6, 1.0, 1.5\}.$$

At each noise level I repeat the Random Forest and CNN evaluation.

The purpose is not to prove that one model is universally noise robust. It is to ask whether the clean-accuracy ranking survives a controlled degradation sweep.

This is particularly useful because the cited literature comparison concerns performance under consumer-grade noise, not only clean synthetic data.

Source: `stage1_signal_processing/phase4_noise_sweep.py`.

5. Stage 2 — real PhysioNet EEG and the leakage investigation

Stage 2 is where the project changes from mathematical sandbox verification to real biological data.

The central dataset is PhysioNet BCI2000 EEG. I use four frontal channels and subject-level evaluation.

The important scripts are:

- `ml_cohort_tournament_v10.py`
- `ml_cohort_tournament_v11.py`
- `ml_cohort_tournament_v34.py`
- `phase1_physionet_validation.py`
- `phase1_diagnostic.py`
- `phase_i_diagnostic.py`
- `batch_physionet_validation.py`
- `subject_normalization_pipeline.py`
- `loso_significance_test.py`

The repository’s reproduction guide explicitly says that v10 and v11 are retained as a case study, while v34 is the final canonical cohort script.

5.1 Common signal-processing logic in v10/v11/v34

The real-data pipeline uses:

- 50-Hz notch,
- 60-Hz notch,
- 1–30-Hz Butterworth bandpass,
- 4 target channels,
- 2-second windows for resting baseline covariance estimation.

The use of `filtfilt` in this offline cohort analysis provides approximately zero-phase filtering. This is appropriate for the offline validation context but should not be confused with a causal real-time filter.

Covariance

Earlier versions used sample covariance plus diagonal regularization:

$$C = \text{Cov}(X) + \epsilon I.$$

v34 upgrades this to Ledoit–Wolf shrinkage:

$$\hat{C} = (1 - \lambda)S + \lambda T,$$

where S is the sample covariance and T is the shrinkage target.

This improves conditioning and makes the SPD representation more stable when covariance estimates are noisy.

6. `ml_cohort_tournament_v10.py` — the important failure case

v10 contains the TSA leakage bug that became one of the strongest verification demonstrations in the project.

The intended idea was to align subject-specific tangent-space features. The problem was that the alignment transform was fitted using **REST-only** tangent vectors and then applied to both REST and ACTIVE classes.

Conceptually, the transform was:

$$A_{rest} = \text{Cov}(T_{rest})^{-1/2}.$$

Applying this same transform to both classes means the coordinate system is deliberately normalized around one class.

Why this is dangerous

Suppose both classes are actually IID Gaussian noise. There is no biological information to classify.

Yet fitting a transformation to one class can make that class artificially compact while making the other class look comparatively dispersed.

The pipeline produced approximately 76.2% LOSO accuracy on pure noise.

That result is not a success. It is a **demonstration of leakage-induced structure**.

Why this became scientifically important

This was the point at which I stopped treating a high accuracy number as evidence of a useful BCI model.

The question became:

“Can the complete pipeline generate above-chance performance when there is mathematically no class information?”

The pure-noise control answered yes for v10.

Therefore v10’s TSA accuracy was not trusted.

Source: `stage2_bci2000_validation/ml_cohort_tournament_v10.py`.

7. `ml_cohort_tournament_v11.py` — fixing the leakage

The key correction was very small in code but very large scientifically.

Instead of:

$$A = \text{fit}(T_{REST}),$$

I changed the fitting set to:

$$A = \text{fit}(T_{REST} \cup T_{ACTIVE}).$$

The transformation therefore represents the subject's overall training distribution rather than one label class.

Why this is the correct conceptual fix

A normalization/alignment operation should not encode the class identity into the coordinate system before classification.

The corrected version was passed through the same pure Gaussian-noise control. The TSA accuracy collapsed to approximately 51.4%, which is consistent with chance.

This is one of the strongest causal chains in the project:

```
v10 high accuracy
  ↓
pure-noise control
  ↓
76.2% accuracy on no-signal data
  ↓
inspect TSA fitting
  ↓
REST-only alignment identified
  ↓
fit on both classes
  ↓
repeat pure-noise control
  ↓
51.4% chance
```

What I learned from this

The negative control did more than validate the final model. It changed the implementation itself.

That is why I regard v10/v11 as part of the scientific contribution of the verification methodology even though v10 is not a valid final result.

Source: `stage2_bci2000_validation/ml_cohort_tournament_v11.py`.

8. `ml_cohort_tournament_v34.py` — final canonical cohort pipeline

v34 is the cleaned canonical version used for the headline cohort result.

It keeps the core mathematical structure from v11 but adds several robustness layers.

8.1 Ledoit–Wolf covariance

For every trial/window:

$$C_{LW} = (1 - \lambda)S + \lambda T.$$

This replaces the simpler regularized sample covariance used earlier.

Why?

The Riemannian pipeline depends on well-conditioned SPD matrices. Shrinkage reduces estimator variance and helps avoid unstable eigenvalues.

8.2 Riemannian mean

The same verified affine-invariant mean logic from Stage 1 is reused:

$$M_{k+1} = M_k^{1/2} \exp \left[\frac{1}{N} \sum_i \log(M_k^{-1/2} C_i M_k^{-1/2}) \right] M_k^{1/2}.$$

The reason for reusing the same mathematical construction is consistency: I do not want the geometry tested in `riemannian.py` to be replaced silently by a different implementation in the real-data pipeline.

8.3 Tangent-space projection

The trial covariance is mapped to:

$$T_C = \log(M^{-1/2} C M^{-1/2}).$$

The symmetric matrix is vectorized with $\sqrt{2}$ scaling for off-diagonal terms.

This produces a Euclidean feature vector suitable for classical ML while retaining the geometry of the SPD representation at the reference point.

8.4 Robust Riemannian outlier detector

For every covariance:

$$d_i = d_R(C_i, M).$$

Then:

$$m = \text{median}(d_i),$$

$$MAD = \text{median}(|d_i - m|),$$

and the modified robust score is:

$$z_i = 0.6745 \frac{d_i - m}{MAD}.$$

I retain trials with:

$$|z_i| \leq 2.5.$$

Why MAD instead of mean/std?

EEG contains outliers. If I used mean and standard deviation, the outliers could distort the very statistics used to detect them. Median/MAD is more robust.

More importantly, the detector works in **Riemannian distance**, so quality control uses the same covariance geometry used by the feature representation.

8.5 Cross-state stationarity gate

I calculate the mean Riemannian distance of REST and ACTIVE covariance pools from a global reference mean:

$$D_{rest} = \frac{1}{N} \sum_i d_R(C_i, M),$$

$$D_{active} = \frac{1}{N} \sum_j d_R(C_j, M).$$

The divergence is:

$$\Delta = |D_{active} - D_{rest}|.$$

A high divergence triggers a conservative raw-feature bypass for the affected subject.

This is an engineering gate rather than a theorem. Its purpose is to prevent an extreme cross-state distribution mismatch from being interpreted blindly as useful classification structure.

8.6 Baseline transport

The tangent-space baseline covariance is estimated from the REST distribution:

$$S_T = \text{Cov}(T_{rest}) + \epsilon I.$$

Its inverse square root gives a transport/whitening matrix:

$$W_T = V \Lambda^{-1/2} V^T.$$

The transformed representation is blended with the original:

$$T' = (1 - \alpha_s)T + \alpha_s T W_T^T.$$

The adaptive α_s is derived from baseline dispersion and clipped to a safe range.

Why this instead of ordinary feature-wise z-scoring?

Feature-wise z-scoring treats each coordinate independently. The tangent coordinates themselves have covariance structure. Whitening the tangent-space covariance attempts to normalize the full second-order feature structure.

I regard the exact adaptive α_s rule as an engineering heuristic, not as a mathematically inevitable consequence of Riemannian geometry.

8.7 Spectral augmentation

Alongside the 10-dimensional symmetric tangent-space representation of a 4×4 covariance matrix, I use 8 channel-wise spectral powers:

- 4 channels × mu (8–13 Hz),
- 4 channels × beta (13–30 Hz).

Thus:

$$10 + 8 = 18$$

features are available to the classical model path.

The motivation is complementary information:

- covariance features represent spatial structure,
 - spectral powers represent channel-wise frequency structure.
-

8.8 Model evaluation

The cohort tournament compares RF, SVM/TSA-related tracks, and the canonical TSA pipeline using subject-level evaluation.

The critical validation structure is **Leave-One-Subject-Out (LOSO)**:

$$Train = \{S_1, \dots, S_{k-1}, S_{k+1}, \dots, S_{50}\}$$

$$Test = S_k.$$

This is much stronger than random trial splitting when the scientific question is generalization to unseen people.

The corrected v34 result reported by the repository is:

- RF Raw: 66.14%
- RF on TSA features: 74.72%
- SVM Raw: 60.39%
- Gated SVM on TSA: 71.96%

with shuffled-label controls near chance.

Negative-control principle

I randomly permute the labels and rerun the complete evaluation. If the pipeline's apparent accuracy were merely an artifact of the feature geometry, it should survive label destruction. It does not: the shuffled TSA baseline is about 47.88%.

That is a much stronger claim than simply reporting 74.72% accuracy.

Source: `stage2_bci2000_validation/ml_cohort_tournament_v34.py`.

9. `subject_normalization_pipeline.py` — asking whether one global baseline is enough

This script studies inter-subject variability in the baseline representation.

The question is:

If I use one global normalization strategy, how differently do subjects behave?

The important idea is that a BCI covariance distribution is subject-dependent. Electrode placement, anatomy, baseline physiology, and recording characteristics can change the covariance geometry.

The script quantifies variability in the EI/generalization-related measures reported in the paper.

This motivated the subject-specific normalization and transport logic in the main cohort pipeline.

The conceptual connection is:

```

subject variability observed
      ↓
need for alignment/normalization
      ↓
Tangent-space transport
      ↓
negative controls + LOSO
      ↓
check that normalization itself does not create class structure

```

Source: `stage2_bci2000_validation/subject_normalization_pipeline.py`.

10. `phase1_physionet_validation.py`, `phase1_diagnostic.py`, and `phase_i_diagnostic.py`

These scripts are diagnostic branches for the real PhysioNet data rather than replacements for v34.

They compare filtered/unfiltered signal references, calculate SNR/correlation diagnostics, and investigate whether preprocessing itself is responsible for apparent similarity or dissimilarity.

The repository explicitly corrected an earlier documentation error: `phase1_diagnostic.py` is **not** the synthetic FastICA/SOBI benchmark. It belongs to the Stage 2 real-data diagnostic branch.

The important principle is reference matching.

If I compare a filtered test signal to a raw reference, part of the discrepancy is simply the filter response. Therefore the diagnostic also considers a similarly filtered reference.

This distinction is important when interpreting:

$$SNR(x_{reference}, x_{test})$$

and:

$$\text{corr}(x_{reference}, x_{test}).$$

The diagnostic branch prevents me from attributing preprocessing-induced differences to the underlying BCI algorithm.

Sources:

- `stage2_bci2000_validation/phase1_physionet_validation.py`
 - `stage2_bci2000_validation/phase1_diagnostic.py`
 - `stage2_bci2000_validation/phase_i_diagnostic.py`
-

11. `batch_physionet_validation.py`

This file provides a batch validation path for the real PhysioNet data. Its role is to execute the real-data preprocessing/evaluation outside the evolving v10→v34 tournament lineage.

Conceptually it performs:

```
PhysioNet EDF
↓
channel selection
↓
filtering
↓
epoch/trial construction
↓
covariance/features
↓
validation metrics
```

I keep this separate from the canonical v34 tournament because a validation/diagnostic script should not silently become the source of the headline number. The repository's `SCRIPT_INDEX.md` identifies v34 as the canonical cohort result.

Source: `stage2_bci2000_validation/batch_physionet_validation.py`.

12. `verification_scripts/loso_significance_test.py`

This is intentionally separate from the model code.

Its input is not raw EEG. Its input is the **per-subject fold accuracies printed by the cohort tournament**.

It parses lines such as:

S01 fold | RF Raw: ... | TSA: ... || SVM Raw: ... | Gated SVM: ...

Then it pairs subjects by ID between intact and shuffled runs.

The paired difference is:

$$d_s = A_{intact,s} - A_{shuffle,s}.$$

The script reports:

- mean paired difference,
- bootstrap confidence interval,
- sign-test counts,
- Wilcoxon signed-rank statistic.

Why Wilcoxon?

The natural experimental unit is the subject, not each EEG window. With 50 subject-level paired observations, I do not need to assume the differences are normally distributed.

The reported canonical result is:

$$W = 66, \quad p \approx 1.36 \times 10^{-7}.$$

This script is deliberately version-agnostic: it can parse the LOSO logs from different tournament versions.

Source: `verification_scripts/loso_significance_test.py`.

13. Stage 3 — closed-loop simulation

Stage 3 asks a different question:

Can the verified computational ideas operate as a streaming closed-loop system rather than only as offline batch analysis?

The files are:

- `virtual_brain_v4_lsl.py`
- `calibration_orchestrator_v2.py`
- `realtime_inference_engine_v2_lsl.py`
- `evaluate_session_v2.py`
- `live_receiver_pipeline.py`

- `lsl_real_time_receiver.py`
- `phase5_calibration_corrected.py`

This is still a **simulation**, not a hardware or human-subject experiment.

13.1 `virtual_brain_v4_lsl.py` — controlled physiological-like signal generator

The simulator produces four-channel EEG-like signals at 250 Hz and publishes them through Lab Streaming Layer (LSL).

The neural sources are stochastic oscillators.

AR(2) oscillator

The model is:

$$x_t = 2r \cos(\omega)x_{t-1} - r^2x_{t-2} + \sigma\epsilon_t.$$

Here:

$$\omega = 2\pi f/f_s.$$

The pole radius r controls persistence/damping, while f controls the resonant frequency.

This is a mathematically controlled way to create oscillator-like signals with stochastic excitation.

Pink-noise floor

The simulator also uses cascaded AR(1)-style states:

$$x_t = px_{t-1} + w_t$$

with multiple poles to create a colored background rather than pure white noise.

State-dependent amplitudes

The simulator has REST, WORKLOAD and MOTOR states. Alpha, theta and beta amplitudes are modulated according to the current state.

Artifacts

The simulator separately generates:

- blink transients,
- muscle artifacts,
- slow drift.

Subject variation

It samples subject-specific parameters such as alpha peak, amplitude scaling, beta reactivity and baseline fatigue.

State control

A Markov state machine can switch states autonomously, while `--manual` allows controlled calibration states.

Why simulation?

Because before building hardware I need to test the software architecture under controlled conditions. The simulator provides known state transitions and known generative mechanisms.

It does **not** prove human BCI efficacy.

Important correction/open-problem lineage

The repository later discovered two incorrect explanations in this simulator study:

1. The 1/f slope issue was not caused by causal-filter group delay. The slope-fit range approached the 45-Hz filter corner; the fit was narrowed from 4–40 Hz to 4–23 Hz.
2. The beta ERD/ERS anomaly was not generic AR(2) cross-frequency coupling. The alpha oscillator damping at $r = 0.93$ allowed substantial alpha power to bleed into the beta range. The tested correction increased damping to $r = 0.99$.

A residual noise-floor/separability trade-off remains an explicitly open problem.

This is a major example of the verification philosophy: **I corrected the causal explanation when the controlled investigation contradicted the original interpretation.**

Source: `stage3_closed_loop_simulation/virtual_brain_v4_lsl.py`.

13.2 calibration_orchestrator_v2.py — corrected Riemannian calibration

This file contains one of the most explicit mathematical bug fixes in the project.

An earlier calibration implementation described a Euclidean average as a Fréchet mean. That was mathematically incorrect.

The correction is:

$$M_{k+1} = M_k^{1/2} \exp \left[\alpha \frac{1}{N} \sum_k \log(M_k^{-1/2} C_k M_k^{-1/2}) \right] M_k^{1/2}.$$

The code uses three 4-minute blocks:

- resting alpha,
- cognitive workload,
- motor imagery.

For each block:

1. receive LSL samples,
2. causally filter 1–45 Hz,
3. maintain a 2-second buffer,
4. calculate SQI,
5. reject poor-quality windows,

6. compute shrinkage covariance,
7. calculate the Riemannian centroid.

The script also calculates pairwise centroid distances:

$$d_R(M_{rest}, M_{cog}), \quad d_R(M_{rest}, M_{motor}), \quad d_R(M_{cog}, M_{motor}).$$

Why Ledoit–Wolf/shrinkage here?

Short windows and only four channels can produce poorly conditioned covariance matrices. Shrinkage stabilizes the SPD object before applying matrix logarithms and inverse square roots.

Why SQI before covariance?

If an artifact-contaminated window enters the calibration mean, the centroid itself becomes contaminated. Therefore signal quality is applied before geometric averaging.

This is another connection to Stage 1:

```
Stage 1 verified geometry
      ↓
Stage 3 uses same SPD/Riemannian principles
      ↓
SQI prevents bad windows from entering the geometric reference
      ↓
centroid distances test state separability
```

Source: `stage3_closed_loop_simulation/calibration_orchestrator_v2.py`.

13.3 realtime_inference_engine_v2_lsl.py

This is the online inference stage.

It consumes the LSL stream, loads the previously saved state centroids, processes incoming windows, computes the current geometric relationship to the calibrated states, and writes session-level signal/metric logs.

The important architectural principle is **separation of calibration and inference**:

```
Calibration
      ↓
fixed reference centroids
      ↓
real-time stream
      ↓
window covariance
      ↓
Riemannian distance/features
      ↓
state inference
```

This prevents the inference process from silently changing its reference geometry every time a new sample arrives.

Source: `stage3_closed_loop_simulation/realtime_inference_engine_v2_lsl.py`.

13.4 `evaluate_session_v2.py`

This script is the post-session analysis layer.

It reads the recorded EEG/metrics CSV files and evaluates quantities such as:

- SQL,
- state separation,
- Cohen's d ,
- Riemannian geodesic distances,
- ERD/ERS-related metrics,
- session-level timing/coverage.

It is deliberately separate from real-time inference. The inference engine should produce the stream-time outputs; the evaluator should independently assess the logged session afterward.

This separation reduces the temptation to modify the live pipeline simply to make a final metric look better.

Source: `stage3_closed_loop_simulation/evaluate_session_v2.py`.

13.5 `live_receiver_pipeline.py`

This is a lightweight receiver path for consuming the streaming EEG feed.

Its role is architectural: receive the LSL stream, maintain a running buffer, and provide data to downstream processing without requiring the simulator itself to know how inference is performed.

It therefore reinforces the separation:

producer $\neq$ receiver $\neq$ inference $\neq$ evaluation.

Source: `stage3_closed_loop_simulation/live_receiver_pipeline.py`.

13.6 `lsl_real_time_receiver.py`

This is the minimal LSL receiver utility used to resolve the EEG stream and pull samples in real time.

It is intentionally small: the purpose is transport/stream acquisition, not signal interpretation.

Source: `stage3_closed_loop_simulation/lsl_real_time_receiver.py`.

13.7 `phase5_calibration_corrected.py`

This file represents the corrected calibration branch associated with the earlier calibration issue. Its purpose is to ensure that calibration parameters are derived from the intended state-specific data and that the downstream real-time system receives a consistent calibration artifact.

I treat it as a correction/lineage artifact rather than as a separate scientific model. The canonical conceptual calibration is the corrected Riemannian centroid procedure documented in `calibration_orchestrator_v2.py`.

Source: `stage3_closed_loop_simulation/phase5_calibration_corrected.py`.

14. Stage 4 — independent human EEG validation

Stage 4 is important because the earlier simulator is controlled by my own generative assumptions.

The STEW dataset gives me an independent source of biological EEG.

The three scripts are:

- `process_stew_benchmark.py`
 - `run_route_a_evaluation.py`
 - `swelling_reproducibility_sweep.py`
-

14.1 `process_stew_benchmark.py` — ingestion and harmonization

The STEW archive contains 14-channel data sampled at 128 Hz.

I ingest the full 48-subject pool, identify resting and workload files, resample:

$$128Hz \rightarrow 250Hz,$$

and select four frontal channels:

$$AF3, F7, F8, AF4.$$

I then remove DC offsets by subtracting each channel's mean.

Why resample?

The rest of my software pipeline operates at 250 Hz. Resampling does not make STEW identical to the PhysioNet data; it simply puts the data into the same computational sampling framework so the downstream analysis can be compared consistently.

Why only four channels?

The objective is not to maximize information from the 14-channel recording. It is to test whether the four-channel computational representation used by the project remains meaningful on an independent human EEG source.

Source: `stage4_stew_crossvalidation/process_stew_benchmark.py`.

14.2 run_route_a_evaluation.py — Euclidean vs Riemannian covariance geometry on human EEG

I window the continuous EEG into 2-second windows with a 1-second step.

Each window becomes a 4×4 covariance matrix:

$$C = \frac{X_c^T X_c}{N - 1} + \lambda I.$$

The diagonal loading:

$$\lambda = 10^{-6}$$

helps guarantee positive definiteness.

I calculate separate Riemannian Fréchet means for resting and cognitive states.

Then I compare the Euclidean mean:

$$M_E = \frac{1}{N} \sum_i C_i$$

with the Riemannian mean:

$$M_R = \arg \min_M \sum_i d_R^2(M, C_i).$$

Determinant-based swelling

The covariance ellipsoid volume is proportional to:

$$\sqrt{\det(C)}.$$

Therefore I examine:

$$\frac{\det(M_E)}{\det(M_R)}.$$

A large ratio indicates that Euclidean averaging has inflated the covariance volume relative to the Riemannian reference.

I also calculate an inter-state geodesic distance:

$$d_R(M_{rest}, M_{cognitive}).$$

Why this is important

The Riemannian claim is no longer tested only on synthetic matrices. The same mathematical distinction is applied to independent biological EEG.

Source: `stage4_stew_crossvalidation/run_route_a_evaluation.py`.

14.3 swelling_reproducibility_sweep.py — retracting a tempting but unstable finding

A single-run swelling experiment showed an interesting non-monotonic behavior around a particular sample/window count.

Instead of treating that pattern as a scientific law, I asked:

Does the dip survive different random subsets?

The script draws 20 independent subsets for each:

$$N \in \{100, 250, 500, 750, 1000, 1500, 2000, 3000\}.$$

For each subset it computes:

$$R_N = \frac{\det(M_E)}{\det(M_R)}.$$

Then it reports mean, standard deviation, coefficient of variation, minimum and maximum.

Why optimize the implementation?

The sweep repeatedly calculates matrix logarithms/exponentials for SPD matrices. General-purpose `scipy.linalg.logm/expm` are more general than necessary. For symmetric matrices I can use eigendecomposition:

$$\log(C) = U \log(\Lambda) U^T,$$

$$\exp(C) = U \exp(\Lambda) U^T.$$

This is mathematically equivalent for the SPD/symmetric case while being much faster for the repeated sweep.

The scientific outcome

The single-run dip was not treated as automatically meaningful. The multi-seed analysis showed that the interpretation needed to be retracted rather than promoted.

This is one of my strongest examples of the principle:

A reproducible negative result is more valuable than an attractive but unstable positive interpretation.

Source: `stage4_stew_crossvalidation/swelling_reproducibility_sweep.py`.

15. The connection between Stage 1 and Stage 4

The relationship is deliberate.

Stage 1

I verify that the Riemannian implementation behaves correctly on a known SPD matrix.

Stage 2

I use the same mathematical construction on real PhysioNet EEG and test whether it produces above-chance subject-level classification.

Stage 3

I use the same covariance/Riemannian concepts in a streaming calibration/inference architecture.

Stage 4

I take the geometry outside the original dataset and apply it to independent human EEG.

So the chain is:

mathematical sanity $\rightarrow$ real EEG $\rightarrow$ streaming simulation $\rightarrow$ independent human EEG

That progression is more important to me than any individual accuracy number.

16. AI narration — `ai_narration_layer/llm_context_generator.py`

The AI narration layer is mathematically different from the EEG pipeline.

Its principle is **constrained generation** rather than post-hoc filtering.

Instead of merely prompting a language model:

“Please avoid unsafe or unapproved structures.”

I define an allowed grammar and reason about the set of outputs that grammar permits.

Conceptually:

$$y \in L(G),$$

where $L(G)$ is the language generated by grammar G .

The guardrail argument is therefore structural: if a forbidden output cannot be generated by the grammar, it is excluded by construction at the grammar layer.

What I do not claim

I do not interpret grammar coverage as a proof that an arbitrary language model is semantically safe.

The repository explicitly describes the 20-prompt result as a **grammar-coverage argument**, not a statistical evaluation of an unconstrained model.

This distinction is important:

syntactic exclusion $\neq$ complete semantic safety.

Source: `ai_narration_layer/llm_context_generator.py`.

17. The most important code lineage in the whole project

If I had to explain the research journey to a professor in one diagram, I would use this:

KNOWN-ANSWER MATHEMATICS

```
SOBI Jacobi sign bug found
  ↓
corrected + known-answer pass

Riemannian geometry sanity check
  ↓
verified SPD mean/tangent mapping

CNN finite-difference gradient check
  ↓
verified backpropagation
```

CONTROLLED SYNTHETIC BENCHMARKS

```
FastICA vs SOBI
heuristic sensitivity
60-trial statistical comparison
AR(4) stochastic benchmark
  ↓
avoid claiming superiority from one easy signal model
```

REAL PHYSIONET EEG

```
v10
  ↓
pure-noise control → 76.2%
  ↓
TSA leakage discovered

v11
```


↓
 fit TSA using both classes
 ↓
 pure-noise control → 51.4%

 v34
 ↓
 robust covariance
 Riemannian outlier filtering
 stationarity gate
 baseline transport
 LOSO
 shuffled-label control
 ↓
 canonical 74.72% result

STREAMING SIMULATION

reuse verified SOBI
 fixed calibration
 causal filtering
 SQI gating
 Riemannian state centroids
 ↓
 closed-loop software validation

INDEPENDENT HUMAN EEG

STEW ingestion
 4-channel harmonization
 covariance geometry
 Riemannian vs Euclidean mean
 multi-seed reproducibility
 ↓
 independent check + retraction of unstable finding

18. Why I did not choose simpler alternatives everywhere

Why not only Euclidean covariance features?

Because covariance matrices are SPD objects with a non-Euclidean geometry. The Riemannian representation tests whether respecting that geometry matters.

Why not only FastICA?

Because FastICA and SOBI exploit different statistical assumptions. Comparing them makes the source-separation choice testable.

Why not only SOBI?

Because a single-method benchmark cannot tell me whether the observed behavior is specific to SOBI or simply a property of the synthetic problem.

Why not raw amplitude features only?

Because raw EEG amplitude is highly subject- and channel-dependent. Covariance captures spatial relationships between channels.

Why not ordinary covariance averaging?

Because Euclidean averaging can produce geometric swelling on SPD matrices. The STEW experiment tests this directly.

Why not ordinary z-score normalization for tangent features?

Because feature covariance matters. The transport approach attempts to account for the full tangent-space covariance structure.

Why not random trial-level train/test splitting?

Because the scientific question is subject generalization. Trial-level splitting can allow subject-specific characteristics into both training and test sets.

Why not use only accuracy?

Because accuracy alone can hide leakage. I therefore add negative controls, subject-level paired statistics, confidence intervals, and reproducibility sweeps.

Why not claim that the simulator is human EEG?

Because it is not. It is a controlled software test environment. Independent human EEG is used separately through STEW.

Why not claim that the CFG proves semantic AI safety?

Because a grammar constrains syntax/allowed structures; it does not prove that every semantically undesirable concept has been excluded.

19. What I consider mathematical principles vs. engineering heuristics

This distinction is important when I explain the project.

Strong mathematical foundations used directly

- PCA/eigendecomposition for whitening and SPD matrix functions
- SOBI/time-lagged covariance
- Jacobi joint diagonalization
- SPD covariance representation
- affine-invariant Riemannian distance

- Fréchet/Karcher mean
- logarithmic tangent-space mapping
- finite-difference gradient checking
- bootstrap resampling
- permutation/sign-flip tests
- Wilcoxon signed-rank test
- LOSO cross-validation
- determinant-based covariance volume
- AR(2)/AR(4) stochastic processes

Engineering/empirical choices

- 20 SOBI lags
- 1–30 Hz offline bandpass
- 2-second covariance windows
- 200-ms streaming update
- MAD threshold 2.5
- cross-state divergence threshold 2.2
- adaptive transport blending range
- simulator damping/noise parameters
- SQI thresholds
- specific frequency bands

I do not want to present the second group as if mathematics uniquely dictated them. They are design parameters that should be justified by the application and, in a future version, subjected to deeper sensitivity/ablation analysis.

20. The open problems I deliberately leave open

My current research does not claim to have solved every engineering problem.

The repository explicitly documents an unresolved noise-floor trade-off in the simulator. Increasing broadband noise can improve one spectral diagnostic while reducing Riemannian state separability below the project’s chosen gate.

There was also an RNG confound: the simulator’s subject profile and noise generation were initially seeded from wall-clock time, meaning parameter comparisons could accidentally compare different simulated subjects. That confound was fixed by pinning seeds, but a noise-floor configuration that reliably satisfies all desired gates has not yet been established.

I regard this as a result, not an embarrassment.

The correct scientific statement is:

The current verification framework exposed a trade-off that the present simulator configuration does not resolve robustly.

That is preferable to selecting one favorable run and declaring the problem solved.

21. Why the code is structured this way

The deeper architecture of my work is therefore:

Implement → Verify → Stress → Apply → Attack with controls → Validate independently → Disclose failures

For example:

SOBI

Implement → known-answer test → synthetic artifact benchmark → AR(4) benchmark → streaming reuse.

Riemannian geometry

Implement → identical-SPD sanity check → real PhysioNet → streaming calibration → independent STEW data.

ML

Build models → gradient-check CNN → cross-validation → shuffled labels → LOSO → subject-level significance.

Simulator

Build AR oscillator → test spectral behavior → investigate unexpected slope/ERD behavior → correct causal explanation → disclose remaining noise-floor trade-off.

Swelling result

Observe interesting effect → reproduce across 20 seeds → find that the single-run interpretation is unstable → retract the interpretation.

That is the central logic of the entire codebase.

22. What I would say verbally when explaining the project

If I have only a few minutes, I would explain it this way:

“I did not start by training a model and asking whether the accuracy looked good. I started by implementing the mathematical primitives I needed and writing known-answer tests for them. SOBI was verified using a controlled source-separation problem, the Riemannian implementation was verified using an identical-SPD test, and the from-scratch CNN was verified by finite-difference gradients. I then moved to synthetic comparisons, where I could control the ground truth. After that I moved to real PhysioNet EEG and deliberately attacked the pipeline with a pure-noise control. That control actually found a leakage bug in v10: it produced 76.2% accuracy from Gaussian noise. I fixed the alignment fitting so both classes contributed to the reference, and the same noise test fell to 51.4%. Only then did I trust the final v34 cohort result. I then tested streaming behavior using the same verified mathematical

primitives and finally used independent human EEG from STEW to test whether the covariance geometry behaved similarly outside the original dataset. Along the way, when a result or explanation failed reproducibility, I corrected or retracted it rather than hiding it.”

That is the shortest accurate description of why the code exists.

23. Final perspective

I consider the current codebase a **verification-first research system**, not a finished hardware BCI.

The current paper establishes the software and mathematical foundation.

The next research stage can build on it with:

```
verified software
  ↓
real EEG hardware
  ↓
causal acquisition
  ↓
real-time artifact rejection
  ↓
real human calibration
  ↓
closed-loop control
  ↓
hardware-constrained evaluation
```

The most important reusable assets from Version 1 are therefore not only the 74.72% number or any single figure. They are:

1. the verified SOBI implementation,
2. the verified SPD/Riemannian implementation,
3. the leakage-audit methodology,
4. the negative-control philosophy,
5. the subject-level evaluation structure,
6. the streaming architecture,
7. the independent human-EEG validation path,
8. and the habit of preserving and documenting failures.

I intend to treat those as the foundation for the next stage rather than as the final destination.

Repository map used for this document

Stage 1

- stage1_signal_processing/sobi.py
- stage1_signal_processing/heuristic_sensitivity.py
- stage1_signal_processing/rms_scaled_injection.py
- stage1_signal_processing/bootstrap_and_permutation.py
- stage1_signal_processing/proper_stats.py

- stage1_signal_processing/ar4_benchmark.py
- stage1_signal_processing/mini_eegnet.py
- stage1_signal_processing/riemannian.py
- stage1_signal_processing/phase2_sqi.py
- stage1_signal_processing/phase3_streaming_ola.py
- stage1_signal_processing/phase4_tournament.py
- stage1_signal_processing/four_way_tournament.py
- stage1_signal_processing/phase4_noise_sweep.py

Stage 2

- stage2_bci2000_validation/ml_cohort_tournament_v10.py
- stage2_bci2000_validation/ml_cohort_tournament_v11.py
- stage2_bci2000_validation/ml_cohort_tournament_v34.py
- stage2_bci2000_validation/batch_physionet_validation.py
- stage2_bci2000_validation/phase1_physionet_validation.py
- stage2_bci2000_validation/phase1_diagnostic.py
- stage2_bci2000_validation/phase_i_diagnostic.py
- stage2_bci2000_validation/subject_normalization_pipeline.py

Stage 3

- stage3_closed_loop_simulation/virtual_brain_v4_lsl.py
- stage3_closed_loop_simulation/calibration_orchestrator_v2.py
- stage3_closed_loop_simulation/realtime_inference_engine_v2_lsl.py
- stage3_closed_loop_simulation/evaluate_session_v2.py
- stage3_closed_loop_simulation/live_receiver_pipeline.py
- stage3_closed_loop_simulation/lsl_real_time_receiver.py
- stage3_closed_loop_simulation/phase5_calibration_corrected.py

Stage 4

- stage4_stew_crossvalidation/process_stew_benchmark.py
- stage4_stew_crossvalidation/run_route_a_evaluation.py
- stage4_stew_crossvalidation/swelling_reproducibility_sweep.py

Statistical verification

- verification_scripts/loso_significance_test.py

AI narration

- ai_narration_layer/llm_context_generator.py

Not covered as computational evidence here

exploratory_not_required_for_paper/ is intentionally excluded. It contains historical development and future deployment/hardware-oriented work rather than scripts required to reproduce the paper's numbered claims.

Important note for discussion

When I explain this work, I should distinguish carefully between:

- **what the mathematics guarantees,**
- **what my controlled experiments demonstrate,**
- **what my real-data experiments support,**
- **what remains an engineering heuristic,**
- **and what is still an open problem.**

That distinction is central to the verification-first philosophy of this project.